MiniCPM5-2B 双国产芯片推理吞吐优化技术报告

——面向天数 BI-V150 与沐曦曦云 C500 的算子、内核与调度优化

赛道：FlagOS 开放计算全球大赛（赛季二）赛道二

队伍：XLANG2026　队员：邓乾（GitHub：xy200303，单人组队）

2026 年 10 月 24 日

摘要

本报告面向国产芯片上的大模型推理吞吐优化任务，在官方指定的 FlagOS 栈（vllm-plugin-FL 分支 flagos-2026-s2、FlagGems 标签 v5.3.5）上，对 MiniCPM5-2B 在天数 BI-V150 与沐曦曦云 C500 两款芯片开展算子、内核与调度优化，保持 `vllm serve` 启动参数、模型与精度要求不变。

性能方面，沐曦 C500 平台 4k 场景总吞吐达到 11868.79–12403.21 tok/s，为官方基线的 2.33–2.44 倍；16k 场景达到 11098.02–11167.87 tok/s，为基线的 1.58–1.59 倍。天数 BI-V150 平台 4k 场景由官网基线 2028.01 提升至 2606.56 tok/s（+28.5%），16k 场景由 915.15 提升至 3541.34 tok/s（+287%）；其中 A20 自研内核在同日交错 A/B 下，相对当日 stock 再提升 16.4% 与 19.8%。精度方面，MATH-500 Level 3 保持在 96.2%–98.1%，高于 95% 的赛题要求；TTFT 在各场景均未回退。

主要优化包括：按输入形状选择 GEMM 与 attention 的实现或执行配置；用 1024-bin 质量直方图实现免排序 top-p 采样；针对 PV 矩阵乘的布局转换开销重写 paged decode attention，使该内核有效带宽由 258 GB/s 提升到 372 GB/s；减少 prefill 阶段的隐式同步与冗余内核发射。

关键词：国产芯片；推理吞吐；算子融合；paged decode attention；shape-aware dispatch；vLLM

目录

1 引言 **3**

1.1 任务定义 3

1.2 评测口径 3

2 总体成绩 **3**

2.1 交付版本与启动验证 4

2.2 性能与约束对照 4

3 优化流程 **6**

4 沐曦 C500：采样链、GEMM 路由与 host 同步消除 **6**

4.1 瓶颈画像 6

4.2 免排序的采样链（opt1 / opt3 / opt7 / opt11） 6

4.3 GEMM 布局与路由（opt2 / opt4 / opt6 / opt8 / opt11） 8

4.4 消除 prefill 每层 DtoH 同步（opt10） 10

5 天数 BI-V150：vendor 接入、发射配置与自研 decode 内核 **12**

5.1 prefill attention 接入 vendor ixAttnBkd（A1 / A6） 12

5.2 decode unified-attention 发射配置重构（A7） 12

5.3 自研 paged decode attention 内核（A20） 13

5.4 批量一致性与路径检查 15

5.5 跨平台结果对照 17

6 正确性验证 **19**

7 合规对照与复现指南 **19**

7.1 合规对照 19

7.2 复现步骤 19

7.3 复现注意事项 21

8 结论 **21**

A 代码与实验记录索引 **22**

B 内核形状扫描与测量波动 **23**

1 引言

在硬件、框架和模型固定的条件下，推理吞吐主要受内核效率、访存布局、host 发射和同步开销影响。本报告以 MiniCPM5-2B[7] 为对象，在天数 BI-V150 与沐曦曦云 C500 上，基于官方指定的 FlagOS 软件栈（vllm-plugin-FL[5] 与 FlagGems[6]）开展优化。

两种芯片在指令集、访存层次、运行时和算子库方面存在差异，通用 GPU 上的常用配置不一定适用。例如，Triton 默认的 `num_warps=4`，`num_stages=3` 在 BI-V150 的 decode attention 上并非合适选择。优化过程先用 profile 和微基准定位瓶颈，再修改内核或执行路径，最后通过官方 benchmark 和 MATH-500 评测端到端效果。后续章节分别介绍 C500 的采样链与 GEMM 路由、BI-V150 的 attention 接入与 decode 内核，以及正确性和复现方法。

1.1 任务定义

本赛题要求在不改变模型、不降低精度的前提下，提升 MiniCPM5-2B 在两指定国产芯片上的推理总吞吐（total tokens/s）。优化必须落在框架与算子内部，且需具备实际开发量：KV Cache 与动态批处理、算子融合、内核重写、自研量化策略等均在允许范围内；直接 merge 上游分支、修改 benchmark 脚本、或在缺乏算子优化的情况下切换算子均被禁止。

1.2 评测口径

表 1 官方性能评测场景与约束

项目	内容
性能指标	total tokens/s（总吞吐，取 4k 与 16k 两场景平均）
4k 场景	[4096, 1024, 64, 256]（输入长、输出长、并发、请求数）
16k 场景	[16384, 1024, 64, 128]
精度要求	MATH-500 Level 3，基线 0.962，要求 ≥ 0.95
TTFT 要求	不得超过基线的 +1%
启动参数	`vllm serve` 参数必须与基线完全一致

本报告所有性能数字均来自官方 benchmark 脚本（`vllm bench serve`）与官方聚合口径，且在无计时埋点、冷缓存条件下测得；精度数字来自 evalscope 官方原命令的 MATH-500 Level 3 子集（105 题）。

2 总体成绩

沐曦 C500 的结果在两台不同实例上各复测一次，取区间报告；天数 BI-V150 的结果以同日交错 A/B 的相对收益表述（该开发实例跨时段漂移较大，跨日绝对值不可比）。

2.1 交付版本与启动验证

交付代码为插件与算子两个独立仓库中的固定提交，均保留完整 Git 历史，可用 `git log`、`git show` 和 `git diff` 查看改动。

表 2 代码版本与记录入口

组件	仓库 / 分支	固定提交与内容
插件	`xy200303/vllm-plugin-FL integration/vllm-plugin-fl`	`0207b7e1`；C500 opt1–opt11、BI-V150 A1–A7、A20 框架 hook
算子	`xy200303/FlagGems integration/flaggems`	`dd574fdd`；采样、GEMM、paged decode attention 等算子
文档	`xy200303/minicpm-flagserve main`	报告、实验台账、复现脚本；不承担运行时代码

2026-10-24 在 C500（64GB 显存、128GB CPU 内存）上，从上述 commit 全新 clone 并安装，按官方命令启动后，服务正常监听 9031 端口，`/v1/models` 返回 `minicpm`，实际 chat 请求正常返回。日志包含 `mcblas_mm` 注册、采样融合启用和 A20 内核启用记录。性能数字来自分平台官方 benchmark。

表 3 总体性能与精度结果

平台	场景	官方基线（tok/s）	本提交（tok/s）	提升
沐曦 C500	4k	5089.645	11868.79–12403.21	2.33×–2.44×
沐曦 C500	16k	7029.675	11098.02–11167.87	1.58×–1.59×
天数 BI-V150	4k	2028.01（官网基线）	2606.56	+28.5%
天数 BI-V150	16k	915.15（官网基线）	3541.34	+287%

精度方面，C500 的 MATH-500 Level 3 为 97.14%–98.1%，天数 BI-V150 为 96.2%–98.1%，均高于 95% 的要求。C500 的 4k TTFT 为 2965 ms（基线 3199 ms），16k TTFT 为 17902 ms（基线 27197 ms）；天数 A20 的 16k TTFT 由 155.5 s 降至 130.1 s。

2.2 性能与约束对照

图 1 汇总吞吐、精度和 TTFT 的相对裕度。吞吐与 TTFT 取相对基线的改善百分比，精度取相对 0.95 门槛的提升幅度。C500 四项数值为 +144% / +58.9% / +2.3% / +34.2%，BI-V150 为 +28.5% / +287% / +1.3% / +16.3%，均满足赛题要求。

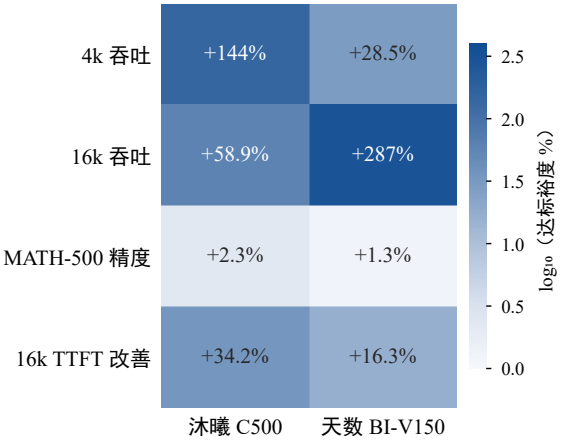
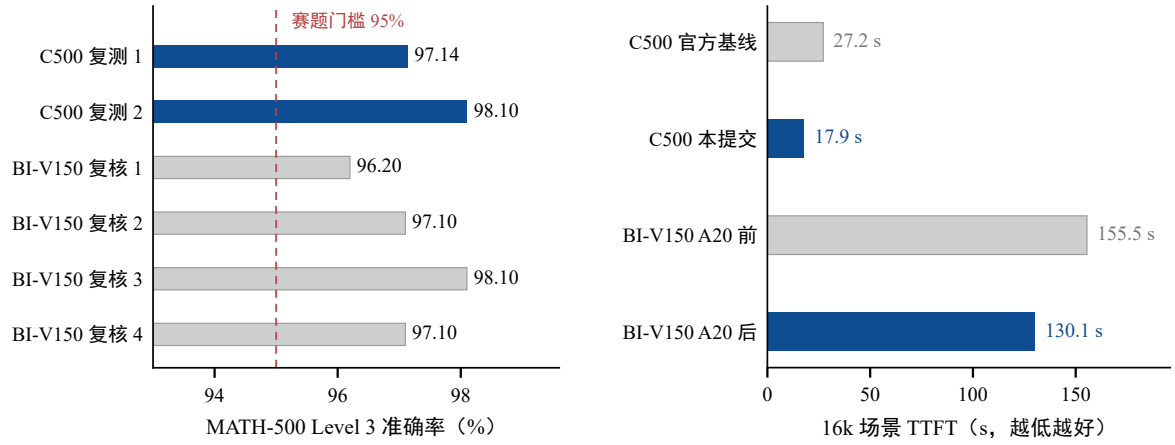


图 1 四项硬约束的达标裕度（相对各自门槛的百分比，颜色取 $\log_{10}$）



(a) MATH-500 Level 3 精度与合格线 (b) 16k 场景 TTFT（越低越好）

图 2 精度与延迟两项约束的对照：四个平台 / 场景组合均达标

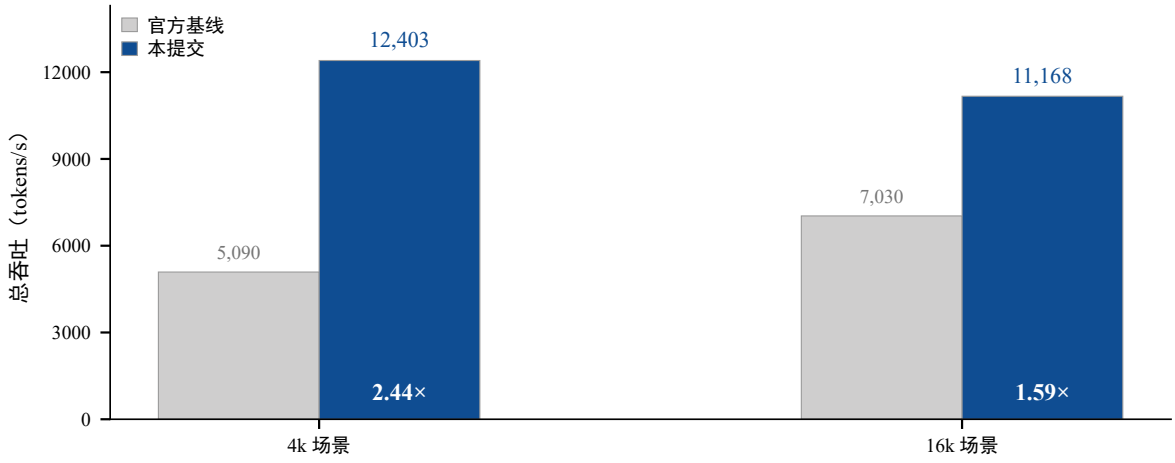


图 3 沐曦 C500：官方基线与优化后总吞吐对比

3 优化流程

两种芯片采用相同的分析与验证流程，测试脚本、profile、日志和 benchmark CSV 按实验编号保存。

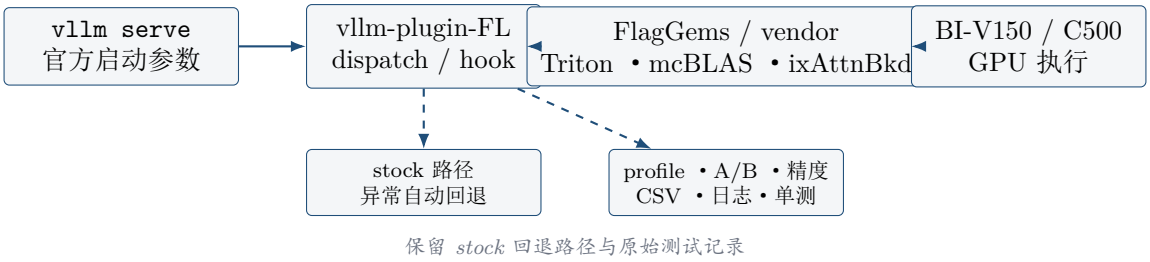


图 4 插件、算子实现与测试记录之间的关系

算子本体位于 FlagGems，框架接入位于 vllm-plugin-FL。性能测试使用官方 benchmark，精度测试使用 evalscope，内核测试与实现文件按附录索引对应。

（1）Profile 定位：结合 serve 时序日志（PagedAttention 式分页 KV 管理见文献 [2]）、设备计时与 host 打点，区分 GPU 内核、host 发射和隐式同步开销；（2）独立测试：新内核先检查目标形状的数值结果，再接入 serve；（3）Shape-aware dispatch：按输入形状选择等价实现或执行配置；（4）端到端测试：保持官方 serve 参数，检查吞吐、TTFT 和 MATH-500 精度。

A6 双 gather、A7 发射配置要求逐位一致；采样融合与 GEMM 路由检查分布和数值误差，并复测 MATH-500。不满足精度要求的改动不进入交付分支。

4 沐曦 C500：采样链、GEMM 路由与 host 同步消除

4.1 瓶颈画像

初始 profile 显示，C500 的主要开销集中在采样、GEMM 和 attention。采样尾部采用 eager PyTorch 实现，包括 bf16→fp32 cast、温度缩放、130560 词表排序和 gumbel-argmax，每步约 12 ms；部分 GEMM 存在布局与路由失配；prefill 的隐式 DtoH 同步则造成 GPU 空闲。

图 5 将同一基准（641 步）两次 profile 的 GPU 内核时间按算子族聚合（源：`docs/baselines/metax-c500/profile_baseline.txt` 与 `profile_post_opt1.txt`）。采样族由 14.61 s 降至 1.24 s，占内核时间的比例由 38.8% 降至 5.1%；GEMM 与 attention 的绝对时间分别为 13.82→13.81 s、7.19→7.22 s。后续优化因此分别针对 GEMM 路由、attention 和 prefill 同步展开。

4.2 免排序的采样链（opt1 / opt3 / opt7 / opt11）

自研 Triton 内核（`FlagGems/fused/sampler_fused.py`、`top_p_sample.py`）做三件事：其一，直接以 bf16 读取 lm_head 输出，把温度折叠进 Gumbel-Max[4]（$\arg\max(x/T+g)$ 与 $\arg\max(x+T\cdot g)$ 等价，$T<10^{-5}$ 时退化为 $\arg\max$）；其二，top-p 用质量直方图 zoom，单轮

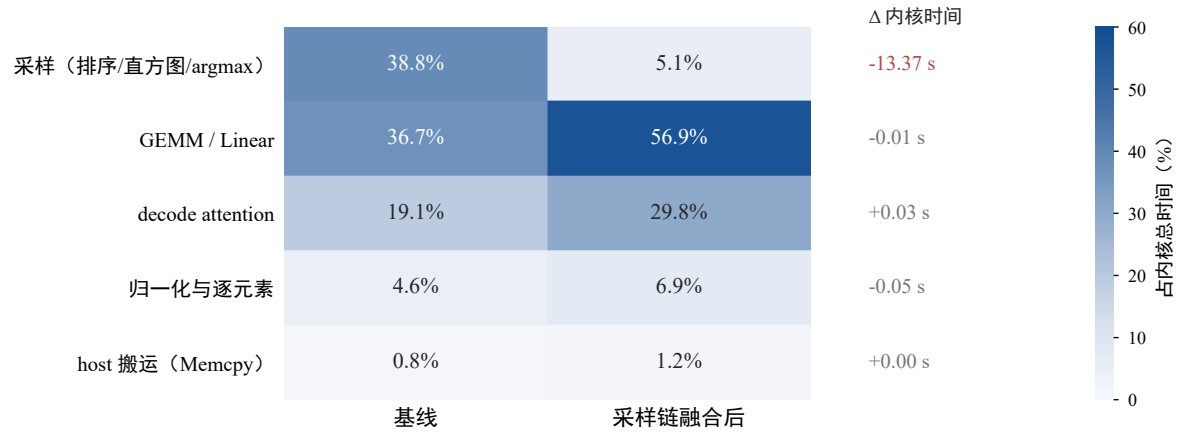


图 5　C500 热路径的算子族构成：融合采样链前后，按内核时间占比（右列为绝对时间变化）

1024-bin 直方图定位阈值 τ 所在 bin 并取其下沿，保留集质量恒不小于 $p \cdot Z$，完全替代全词表排序；其三，词表维 split-parallel（32 splits × BLOCK 4096），使小 batch 也能打满芯片。采样全链由约 12.4 ms 降至 0.31 ms/步。

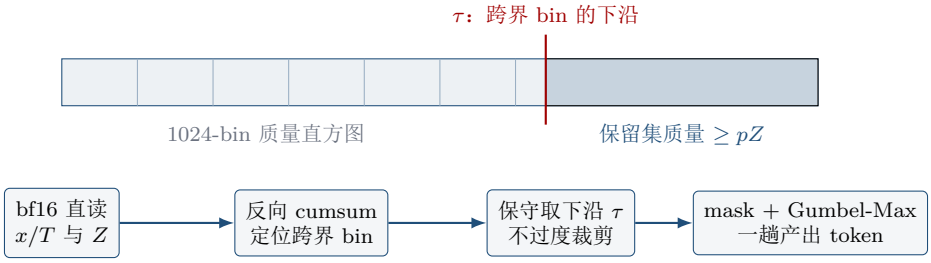


图 6　免排序 top-p 采样链示意图（流程与阈值规则为代码语义，非另造的分布数据）

图 6 中的阈值取所在 bin 的下沿，保证保留集质量不小于 pZ；采样分数写成 $x + Tg$，与先算 x/T 再加 g 的 argmax 等价。对应实现如下。

top_p_sample.py:180--198：反向 cumsum 定位阈值并保守取下沿

```
180     mass_rev = tl.load(HIST + row * NBINS + (NBINS - 1 - offs))
181     cum = tl.cumsum(mass_rev, axis=0)
182     z = tl.load(Z_ROW + row)
183     p = tl.load(P + row).to(tl.float32)
184     lo = tl.load(LO + row)
185     hi = tl.load(HI + row)
186     w = (hi - lo) / NBINS
187
188     target = p * z
189     reach = cum >= target
190     # crossing bin b*: cum-from-top first reaches target there;
191     # sum(reach) == b* + 1 (reversed positions r*..NBINS-1 all reach)
192     b = tl.sum(reach.to(tl.int32), axis=0) - 1
193     b = tl.minimum(tl.maximum(b, 0), NBINS - 1)
194     # conservative: tau = lower edge of crossing bin => mass{x>=tau} >= target
195     new_lo = lo + b * w
196     new_hi = new_lo + w
197     if FINAL:
198         tl.store(TAU + row, new_lo)
```

top_p_sample.py:241--249：温度折叠的 Gumbel-Max 与 top-p mask

```
241         keep = xs >= tau
242         u = tl.rand(row_seed, step * vocab_padded + idx.to(tl.int32))
243         u = tl.minimum(tl.maximum(u, 1e-20), 1.0 - 1e-7)
244         g = -tl.log(-tl.log(u))
245         score = tl.where(keep, x + noise * g, float("-inf"))
246         tile_best = tl.max(score, axis=0)
247         tile_arg = tl.argmax(score, axis=0)
248         take = tile_best > best
249         best_idx = tl.where(take, start + tile * BLOCK + tile_arg, best_idx)
```

正确性方面，greedy 与 `torch.argmax` 完全一致；top-p 样本全部落在核集内（含直方图粒度 slack），与原实现的 TV 距离处于纯采样噪声区间。

4.3 GEMM 布局与路由（opt2 / opt4 / opt6 / opt8 / opt11）

MACA Triton 的 `mm_kernel` 只支持 nn 布局。在 prefill 大 M 的 126 组配置扫描中，其耗时为 vendor mcBLAS 的 1.35–1.67 倍，后者约为 197 TFLOPS。FlagGems 接管 `aten::mm` 后，原路由无法直接调用 vendor GEMM。为此，prefill 大 M 通过 ctypes 调用 libmcblas 的 `mcblasGemmEx`（cuBLAS 兼容 ABI），并注册为 custom op 以适配 compiled 区域；decode 细瘦 M 使用 Triton nn_db 单布局内核，权重只保留一份。另修复 `vocab_parallel_embedding` 在 import 时绑定原版 `dispatch_unquantized_gemm` 导致 lm_head 未进入优化路由的问题，lm_head 由 603 μs 降至 379 μs。

只有已经生成 nn 副本且形状满足条件的线性层才进入新路径；小 M 或没有副本时仍走原

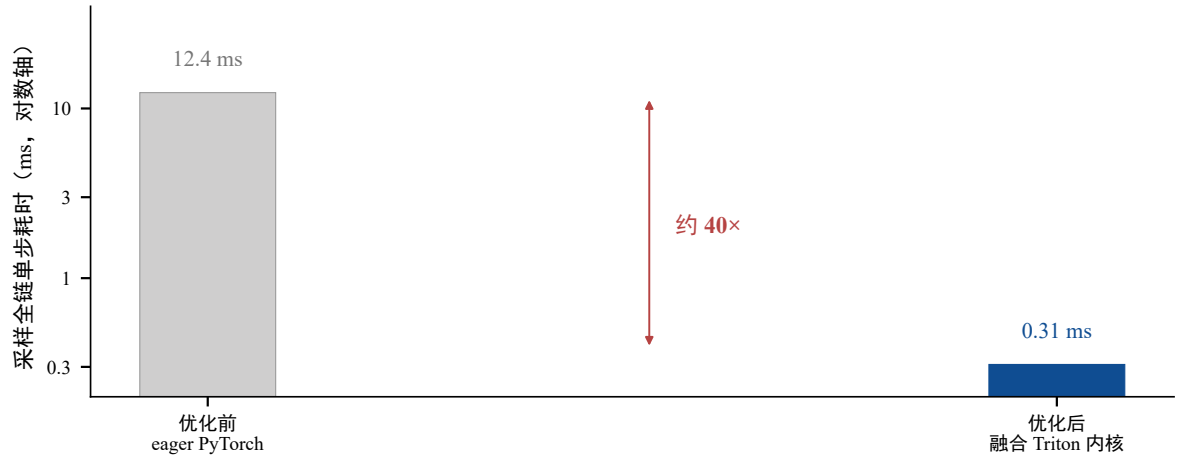


图 7　采样全链单步耗时（对数坐标）

F.linear。lm_head 通过独立的 _fl_always_nn 标志使用 nn 副本，避免被通用 N 上限排除。

linear_nn_repack.py:112--129：按形状在 vendor、Triton 与 stock 之间路由

```
112 def _metax_unquantized_gemm(layer, x, weight, bias):
113     wn = getattr(layer, "_fl_w_nn", None)
114     if wn is not None and bias is None and x.dim() >= 2:
115         m = x.numel() // x.shape[-1]
116         if (
117             getattr(layer, "_fl_always_nn", False)
118             or m > _FL_NN_MIN_M
119             or weight.numel() == 0
120         ):
121             x2 = x.reshape(-1, x.shape[-1])
122             # Vendor mcBLAS beats the Triton nn kernel 1.35-1.67x at prefill
123             # M (sweep_mm_prefill.py); FlagGems' process-wide aten override
124             # makes the vendor library unreachable through torch.mm, so the
125             # ctypes-bound custom op is the only way to reach it.
126             if (
127                 _mcblas is not None
128                 and _mcblas.AVAILABLE
129                 and wn.dtype == torch.bfloat16
```

ctypes 绑定把行主序的 $C = AB$ 转写成列主序的 $C^T = B^T A^T$，因此不需要改变模型的数学结果；register_fake 让编译器在 fake tensor 阶段仍能推导输出形状。

mcblas_mm.py:85--103：直接调用 mcBLAS，保持原 GEMM 语义

```
85  def _mm_impl(a: torch.Tensor, b: torch.Tensor) -> torch.Tensor:
86      M, K = a.shape
87      K2, N = b.shape
88      assert K == K2
89      out = torch.empty(M, N, device=a.device, dtype=torch.bfloat16)
90      st = _lib.mcblasSetStream(
91          _handle, ctypes.c_void_p(torch.cuda.current_stream().cuda_stream)
92      )
93      assert st == 0, f"mcblasSetStream failed: {st}"
94      st = _lib.mcblasGemmEx(
95          _handle, _MCBLAS_OP_N, _MCBLAS_OP_N,
96          N, M, K,
97          ctypes.byref(_alpha),
98          ctypes.c_void_p(b.data_ptr()), _MACA_R_16BF, N,
99          ctypes.c_void_p(a.data_ptr()), _MACA_R_16BF, K,
100         ctypes.byref(_beta),
101         ctypes.c_void_p(out.data_ptr()), _MACA_R_16BF, N,
102         _MCBLAS_COMPUTE_32F, _MCBLAS_GEMM_DEFAULT,
103     )
```

4.4 消除 prefill 每层 DtoH 同步（opt10）

prefill trace 中 GPU 空闲 18.1%，元凶之一是 vendor flash_attn 每层每步执行 `prefill_seq_lens.tolist()`（DtoH 强同步，42 层各一次）。改为纯 GPU cumsum 并缓存到当步 `attn_metadata`，42 层只算一次：4k 提升 10.9%、16k 提升 9.9%，16k TTFT 由 20.8 s 降至 18.0 s。

flash_attn.py:790--810：把每层一次的 DtoH + cumsum 改成当步缓存

```
                    attn_metadata,
                    q_descale=layer._q_scale.expand(descale_shape),
                    k_descale=layer._k_scale.expand(descale_shape),
                    v_descale=layer._v_scale.expand(descale_shape),
                )
                return output
            else:
                # ------------------------  Metax Modification -------------------------
                # For handling prefill decode split
                num_decode_tokens = attn_metadata.num_decode_tokens
                if attn_metadata.num_prefills > 0:
                    # fl opt10: compute once per step on GPU (the old code did
                    # tolist()->DtoH sync + HtoD + cumsum PER LAYER = 42 syncs/step)
                    cu_prefix_kv_lens = getattr(
                        attn_metadata, "_fl_cu_prefix_kv_lens", None
                    )
                    if cu_prefix_kv_lens is None:
                        lens = attn_metadata.prefill_seq_lens
                        cu_prefix_kv_lens = torch.cat(
                            [lens.new_zeros(1), lens]
                        ).cumsum(dim=0, dtype=torch.int32)
```

这段代码只改变 metadata 的生产方式，不改变请求组成、调度顺序或 `vllm serve` 参数；缓存对象仍然属于当前 `attn_metadata`，不会跨请求复用旧长度。

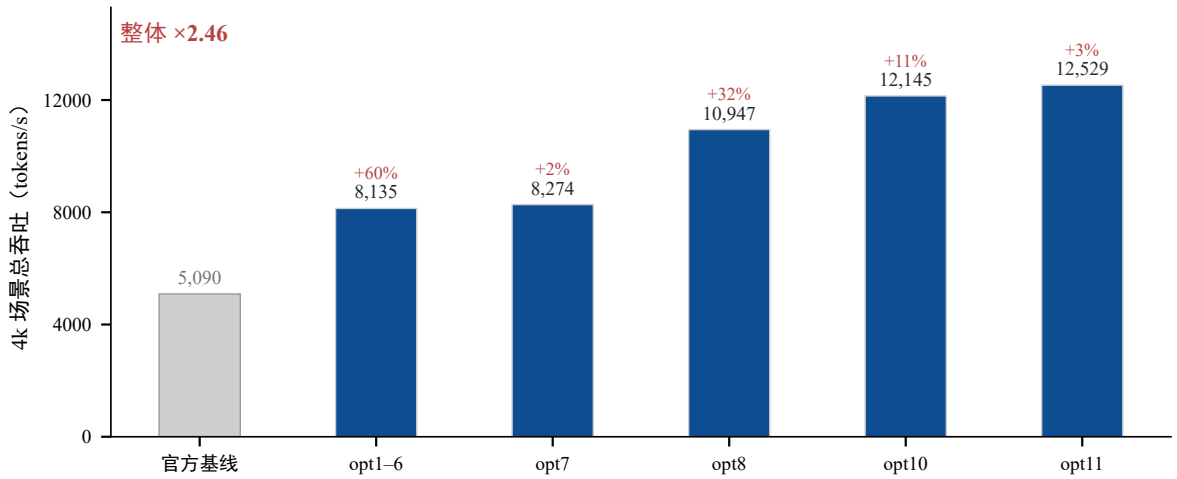


图 8 C500 4k 场景优化阶梯（total tokens/s）

5 天数 BI-V150：vendor 接入、发射配置与自研 decode 内核

5.1 prefill attention 接入 vendor ixAttnBkd（A1 / A6）

iluvatar 后端缺少 flash_attn 包时，attention 落到 vLLM 通用 Triton 参考实现，16k prefill 每层每 2048-chunk 需 36.8 ms，占 GPU busy 91.8%。接入 vendor ixAttnBkd 内核并确认其 causal 语义为 bottom-right（cmt=2）与 vLLM 的 chunk prefill 对齐后，内核级实测在所有 kv 长度上均快 12-20 倍。配套地，builder 一次性完成每步 decode/prefill 划分，避免每层两次 D2H；KV 用自研 Triton `_dual_gather_kernel` 一次完成 K+V 分页收集（bit-exact，GPU 106→62 μs、host 68→37 μs），varlen 分支用 out= 直写消除 copy-out，每请求发射数由 4 降为 2。

vendor_flash_attn.py:46--58：同一 Triton program 收集一页 K 与 V

```
    def _dual_gather_kernel(KVC, PAGES, KP, VP,
                            E: _tl.constexpr, BLOCK: _tl.constexpr):
        # kv_cache [blocks, 2, bs, hkv, d] flat: page p's K chunk at
        # p*2*E, V chunk right after.  One program per page copies both.
        pid = _tl.program_id(0).to(_tl.int64)
        page = _tl.load(PAGES + pid)
        offs = _tl.arange(0, BLOCK)
        src = KVC + page * (2 * E)
        dst = pid * E
        for i in range(0, E, BLOCK):
            _tl.store(KP + dst + i + offs, _tl.load(src + i + offs))
            _tl.store(VP + dst + i + offs, _tl.load(src + E + i + offs))
except Exception:
```

其中 `E=bs*hkv*d` 是一页 K 的元素数，V 的源地址位于 K 之后的连续区域。内核利用这一布局在同一 program 中收集 K 和 V，数值测试与两次 `index_select` 逐位一致。

5.2 decode unified-attention 发射配置重构（A7）

stock Triton 默认 `num_warps=4, num_stages=3`。扫描结果表明，该配置在 BI-V150 上并不合适，软件流水线在各 warp 档上均未带来收益。对 3D 主内核与 reduce_segments 全链路扫描后，将 3D flash-decoding 路径固定为 w1 s1，保持 TILE=16、SEG=16，2D 路径不变；输出与 stock 逐位一致。

decode_attn_pin.py:35--47：只包住 3D 内核的发射配置，2D 路径不变

```
    class _PinnedLaunch:
        def __getitem__(self, grid):
            fn = kern[grid]

            def _call(**kw):
                if kw.get("IS_3D"):
                    kw["num_warps"] = _NUM_WARPS
                    kw["num_stages"] = _NUM_STAGES
                return fn(**kw)

            return _call

    _tua.kernel_unified_attention = _PinnedLaunch()
```

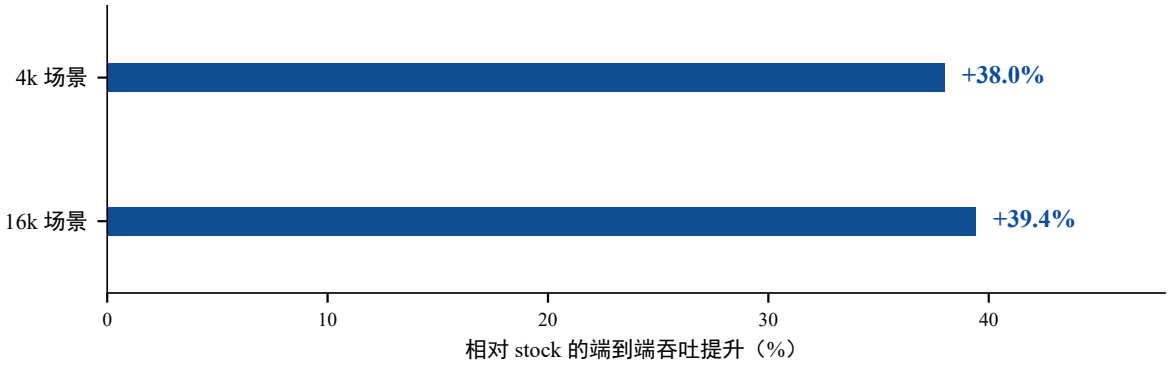


图 9　A7 发射配置重构带来的端到端吞吐提升

5.3　自研 paged decode attention 内核（A20）

A7 之后，decode attention 仍占整步的主要时间。在同一 harness 下逐段测量 stock kernel（B=64 / kv=4096 / 16q-2kv / d=128 / block=16 / bf16）：loads+QK 的有效带宽为 509 GB/s，PV 的 `tl.dot` 耗时 0.36 ms。PV 需要将 p 从 MMA 累加器布局转换为 A 操作数布局，每个 KV block 都产生共享内存往返，因此本阶段的主要开销是布局转换。

据此重写内核（算子位于 `FlagGems/fused/paged_decode_attention.py`，插件侧为薄封装与派发）：一个 CTA = 1 warp =一个（seq, kv_head, 段），使布局转换留在 warp 内；BLOCK_N=16，p tile 不超过一个 MMA 形状；base-2 在线 softmax，scale 预折 $\log_2(e)$ 走 MUFU；主循环不加 mask，仅 ragged 尾部单独一个带 mask 的 block；段数固定、段长由 device 端 seq_lens 现算（CUDA-graph 安全），段缓存 slab 在 `apply()` 即捕获前一次性预分配；段边界按整页对齐，进一步消去逐 token 除模与向量页 gather。混合步中的 decode 行也通过 `try_fast_decode` 进入同一内核。

下面的主循环展示了三个直接针对瓶颈的选择：页边界对齐后 `page` 只需标量加载；`exp2` 代



主循环无 mask；仅 ragged 尾块 mask；段长 device 端计算，slab 在图捕获前预分配

图 10 A20 paged decode attention 的分段执行与归并流程

替 `exp`；每段只写自己的 `SEG_ACC/SEG_MAX/SEG_SUM`，最后由 combine 按 $\exp 2(m_i - m)$ 加权归并。

`paged_decode_attention.py:169--193`：页对齐主循环与 base-2 在线 softmax

```
169     for t0 in range(start, end, BS):
170         page = tl.load(BT + pid_s * stride_btb + (t0 // BS) * stride_btp)
171         if t0 + BS <= kvlen:
172             kt = tl.load(K + page * stride_kb + pos[None, :] * stride_ks
173                          + pid_h * stride_kh + offs_d[:, None] * stride_kd)
174             vt = tl.load(V + page * stride_vb + pos[:, None] * stride_vs
175                          + pid_h * stride_vh + offs_d[None, :] * stride_vd)
176             s = tl.dot(qv, kt) * scale
177         else:
178             mask_t = t0 + pos < kvlen
179             kt = tl.load(K + page * stride_kb + pos[None, :] * stride_ks
180                          + pid_h * stride_kh + offs_d[:, None] * stride_kd,
181                          mask=mask_t[None, :], other=0.0)
182             vt = tl.load(V + page * stride_vb + pos[:, None] * stride_vs
183                          + pid_h * stride_vh + offs_d[None, :] * stride_vd,
184                          mask=mask_t[:, None], other=0.0)
185             s = tl.where(mask_t[None, :], tl.dot(qv, kt) * scale,
186                          float("-inf"))
187         m_new = tl.maximum(m_i, tl.max(s, 1))
188         alpha = tl.exp2(m_i - m_new)
189         p = tl.exp2(s - m_new[:, None])
190         l_i = l_i * alpha + tl.sum(p, 1)
191         acc = acc * alpha[:, None] + tl.dot(p.to(vt.dtype), vt)
192         m_i = m_new
193
```

paged_decode_attention.py:207--223：分段结果的在线归并

```
    pid_s = tl.program_id(0)
    pid_h = tl.program_id(1)
    offs_g = tl.arange(0, GQA)
    offs_d = tl.arange(0, D)
    offs_n = tl.arange(0, NSEG)

    base = (pid_s * HKV + pid_h) * NSEG
    mx = tl.load(SEG_MAX + (base + offs_n)[None, :] * GQA + offs_g[:, None])
    sm = tl.load(SEG_SUM + (base + offs_n)[None, :] * GQA + offs_g[:, None])
    m = tl.max(mx, 1)
    w = tl.exp2(mx - m[:, None])
    denom = tl.sum(w * sm, 1)
    accs = tl.load(SEG_ACC + ((base + offs_n)[None, :, None] * GQA + offs_g[:, None, None]) * D
                   + offs_d[None, None, :])
    out = tl.sum(accs * w[:, :, None], 1) / denom[:, None]
    tl.store(OUT + pid_s * stride_ob + (pid_h * GQA + offs_g)[:, None] * stride_oh
             + offs_d[None, :] * stride_od, out.to(OUT.dtype.element_ty))
```

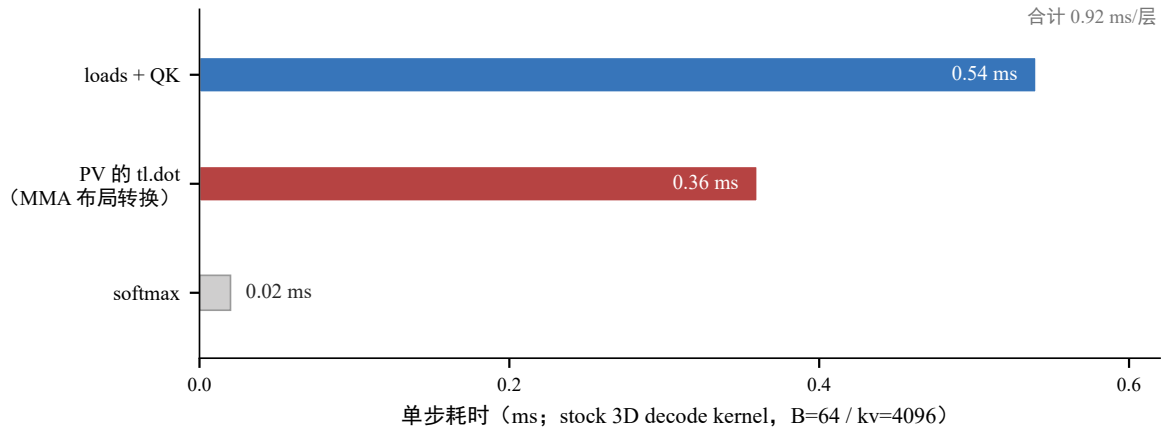


图 11 stock 3D decode 内核的耗时分解：瓶颈在 PV 的 MMA 布局转换

内核级结果为 1.04 → 0.72 ms/层（ragged 长度下 0.90 → 0.66 ms），输出与 stock 的 max|diff| 为 2.4×10^{-4}（ragged 为 9.8×10^{-4}）。同日交错 A/B：4k 由 2239.43 提升至 2606.56 tok/s（+16.4%），16k 由 2956.59 提升至 3541.34 tok/s（+19.8%）。精度 MATH-500 Level 3 四次复核为 96.2% / 97.1% / 98.1% / 97.1%，均达标。

5.4 批量一致性与路径检查

每个（seq, kv_head, segment）只依赖自身的 q、KV 页和 seq_lens，段长仅由序列长度决定。`tests/test_paged_decode_attention.py` 包含 8 个 fp32 参考对照用例和一个批量不变性用例，在 BI-V150 上共 9 passed。批量测试覆盖单条、两条、半批、全批和逆序全批，并重复执行同一批组成。混合步的 decode 切片每 4000 次调用记录 used/skipped；两次 16k 实测均为

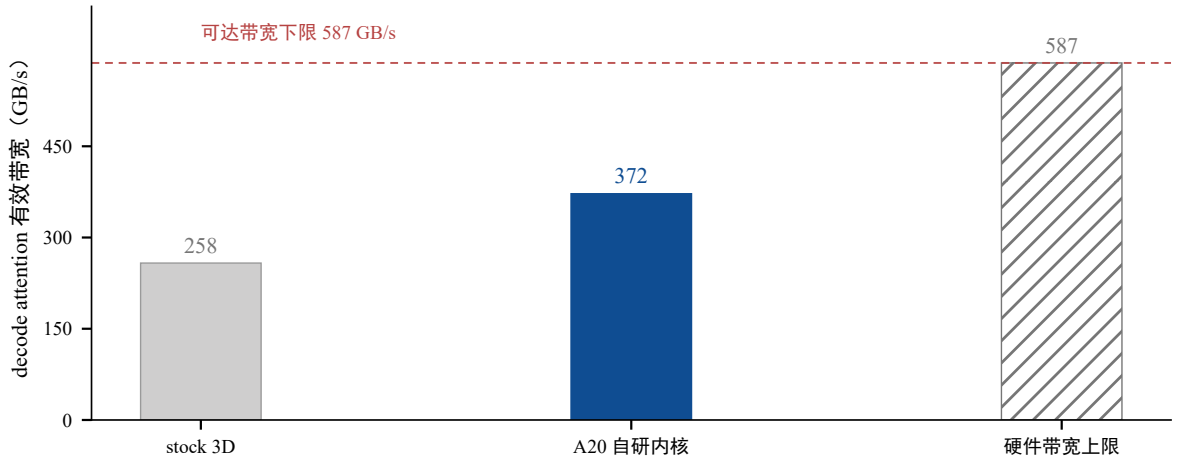


图 12 decode attention 有效带宽对比（kv=4096，B=64）

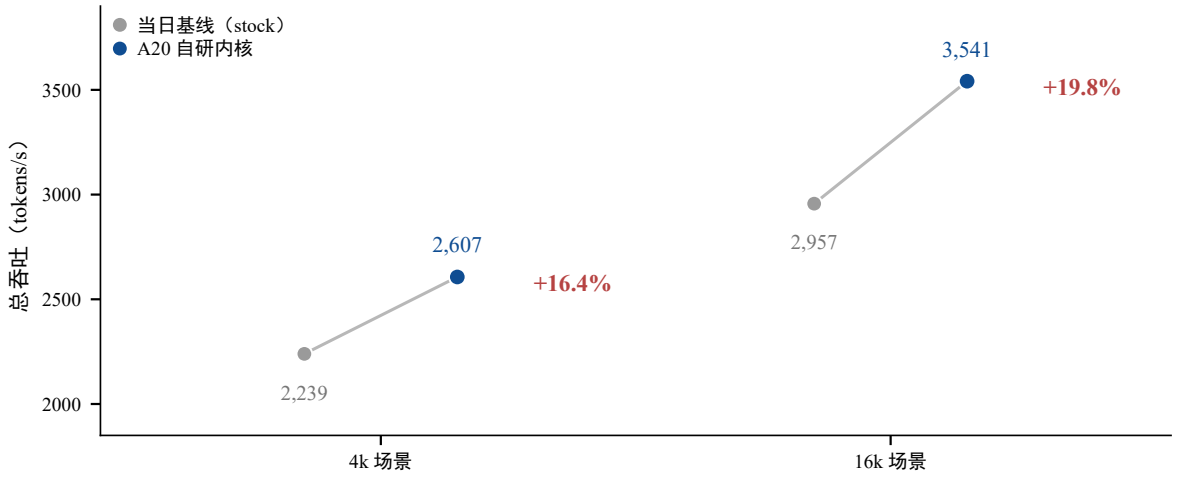


图 13 天数 BI-V150：A20 同日交错 A/B 结果

used=44000、skipped=0。

decode_attn_fast.py:186--205：混合步 decode 切片的资格检查、计数与异常回退

```
186 def try_fast_decode(impl, query, kv_cache, md, out):
187     """Entry point for the mixed-step (prefill+decode) path.
188 
189     Returns True when the fast kernel computed `out` for the decode slice.
190     A periodic INFO line reports the engagement ratio so a run can be
191     audited from its serve log alone.
192     """
193     if _DISABLED or md is None:
194         return False
195     if not _eligible(impl, query, kv_cache, md, None):
196         _slice["skip"] += 1
197         _slice_report()
198         return False
199     _slice["used"] += 1
200     _slice_report()
201     try:
202         return _fast_forward(impl, query, kv_cache, md, out) is not None
203     except Exception as exc:
204         logger.warning("decode_attn_fast: slice fallback (%s)", exc)
205         return False
```

满足形状与特征条件时进入新内核；遇到不支持的特征或异常则回到原实现。used/skipped 计数用于检查混合批中的实际执行路径。

5.5 跨平台结果对照

图 14 对比两种芯片的收益构成。C500 按累计版本展示 4k 吞吐变化；BI-V150 的 A6、A7、A20 使用各自的当日对照，三组数据不作累计。

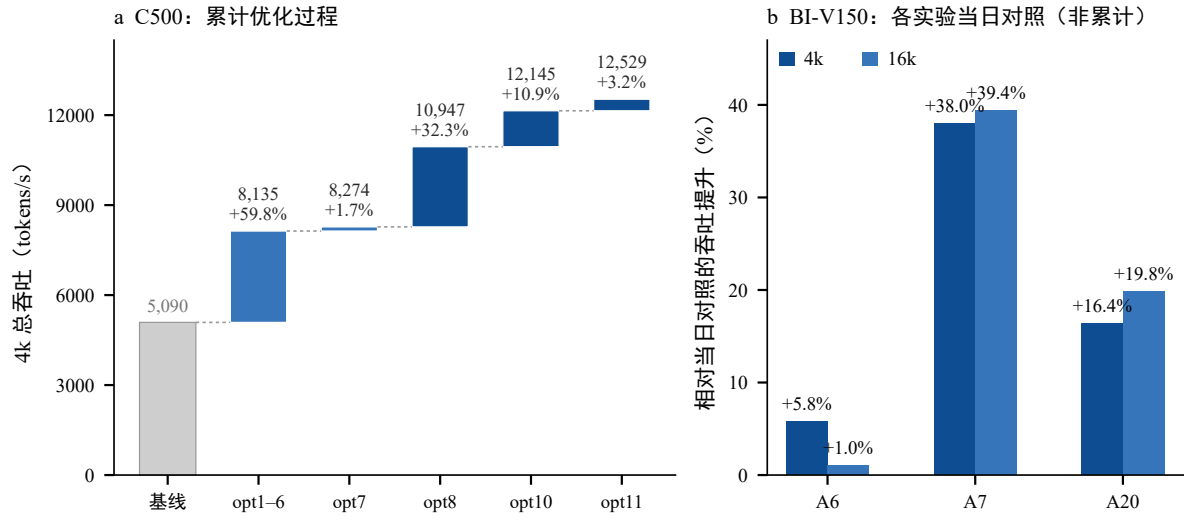


图 14 优化阶段的吞吐收益：C500 为累计过程，BI-V150 为各实验的当日对照

局部加速与端到端收益的关系见图 15。其中 opt8 的横坐标为四种 prefill GEMM 形状加速比的算术平均值，A20 为 stock 1.04 ms 与新内核 0.72 ms 之比。由于各内核在完整模型中的时间占比不同，局部加速比与端到端提升之间不存在固定的线性换算关系。

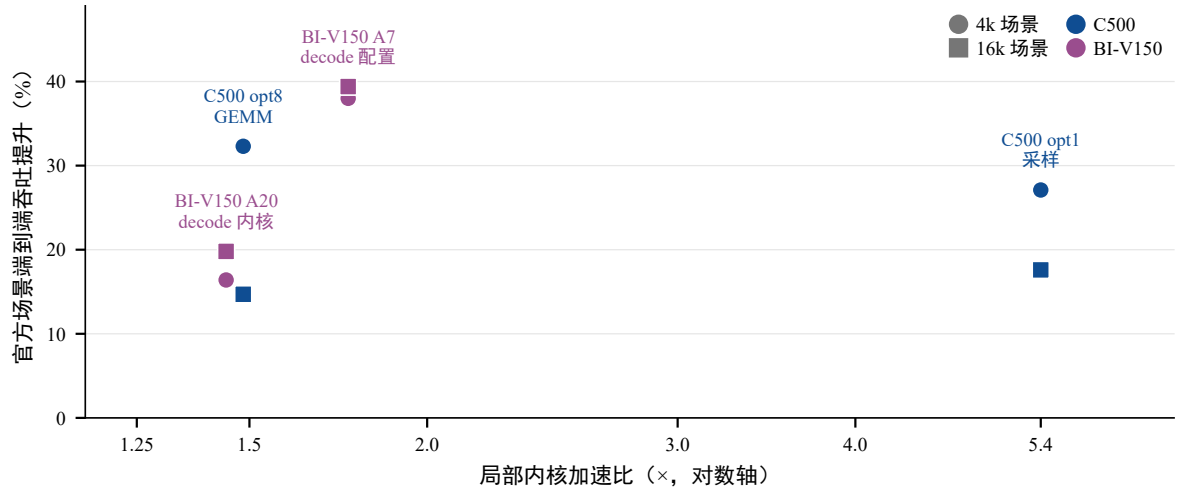


图 15　局部内核加速比与官方场景端到端吞吐提升的关系

图 16 进一步比较内核效率。左图将 decode attention 带宽分别归一到同芯片的实测参考：BI-V150 使用 copy 587 GB/s，C500 使用 sum 1610 GB/s；两种参考不作为跨芯片的绝对性能排名。右图给出 C500 四种 prefill GEMM 形状的计算吞吐，Triton 数值由相同矩阵运算量与实测耗时计算。

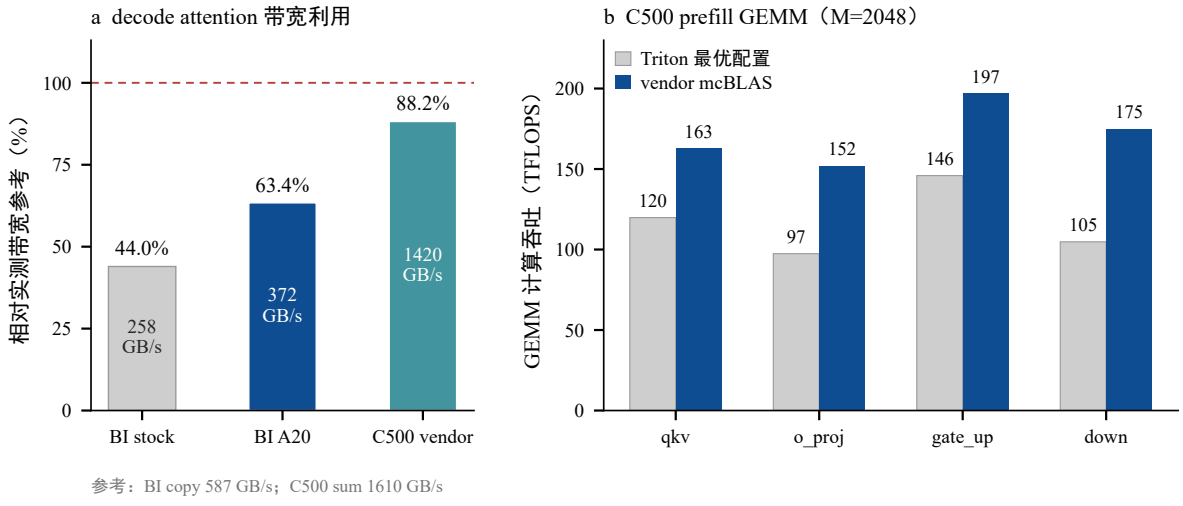


图 16　decode attention 带宽利用与 C500 prefill GEMM 计算吞吐

6 正确性验证

正确性检查覆盖内核数值、批量不变性、采样分布和端到端精度。各层的测试对象与结果见下表。

表 4 正确性验证矩阵：从单个 kernel 到完整服务

验证层	对象与方法	实测结论
bit-exact	A6 双 gather、A7 发射配置	A6 与两次 `index_select` 逐位一致；A7 与 stock 的 `max_diff`=0.0
数值误差	A20 vs stock，含 ragged 形状	4k 最大 2.4×10^{-4}，ragged 最大 9.8×10^{-4}
批量不变性	单条 / 两条 / 半批 / 全批 / 逆序全批，对照 fp32	9 个用例通过；同组成重复执行结果一致
采样等价	greedy、top-p、Gumbel 与 TV 距离	greedy 与 `torch.argmax` 完全一致；保留集 IoU ≥ 0.999；TV 与参考实现同档
端到端精度	evalscope MATH-500 Level 3（105 题）	C500 97.14%–98.1%；BI-V150 96.2%–98.1%，均超过 95%
真实路径命中	混合步 decode 切片 used/skipped	两次 16k 均为 used=44000、skipped=0，无隐式回退
交付可运行	全新 clone + 独立 venv + 官方启动	C500 启动、`/v1/models` 与实际 chat 请求均通过

7 合规对照与复现指南

7.1 合规对照

表 5 赛题约束与本提交做法

约束	本提交做法
`vllm serve` 参数与基线一致	两张卡均按官网原命令启动，未增删任何参数；C500 不额外添加 `--compilation-config`
禁止 benchmark 脚本调优	一律使用官方脚本与官方聚合口径，未修改脚本
禁止直接 merge 上游分支	基线为 flagos-2026-s2 与 FlagGems v5.3.5，全部改动以补丁形式叠加
算子优化须有开发量	新增 Triton 内核（paged decode attention、采样链、dual-gather）与路由/同步改造，均可独立 A/B
禁止改超参开启量化、投机采样	W8 权重量化与 n-gram 投机均在评估后未采用，未进入提交
精度 ≥ 0.95	MATH-500 Level 3：C500 97.14%–98.1%，BI-V150 96.2%–98.1%
TTFT 不回退	各场景 TTFT 均优于基线；16k C500 由 27197 ms 降至 17902 ms

7.2 复现步骤

先取得两个集成分支（保留完整历史），再分别以可编辑模式安装：

源码获取与安装

```
git clone --single-branch --branch integration/vllm-plugin-fl \
  https://github.com/xy200303/vllm-plugin-FL.git vllm-plugin-FL
git clone --single-branch --branch integration/flaggems \
  https://github.com/xy200303/FlagGems.git FlagGems

cd vllm-plugin-FL && pip install -e . --no-build-isolation
cd ../FlagGems   && pip install -e . --no-build-isolation
```

沐曦 C500 还需设置以下环境变量（天数无需）：

C500 环境变量（天数无需额外设置）

```
export PATH=/opt/conda/bin:$PATH
export MACA_PATH=/opt/maca
export MACA_HOME=/opt/maca
export LD_LIBRARY_PATH=/opt/maca/lib:/opt/maca/lib64
export VLLM_PLUGINS=fl
```

启动命令与官网完全一致，天数在官方命令后追加编译配置，C500 不追加：

官方启动命令（与基线一致）

```
# 天数 BI-V150
vllm serve /workspace/MiniCPM5-2B --port 9031 --served-model-name minicpm \
  --gpu-memory-utilization 0.85 --max-model-len 131072 \
  --compilation-config '{"cudagraph_mode": "FULL_DECODE_ONLY"}'

# 沐曦 C500（不添加 --compilation-config）
vllm serve /workspace/MiniCPM5-2B --port 9031 --served-model-name minicpm \
  --gpu-memory-utilization 0.85 --max-model-len 131072
```

性能评测执行官方脚本并传入两个官方用例：

官方性能评测命令

```
python3 vllm-plugin-FL/benchmarks/benchmark_throughput_serve.py \
  --model /workspace/MiniCPM5-2B --served-model-name minicpm --port 9031 \
  --test-cases '[[4096,1024,64,256],[16384,1024,64,128]]'
```

精度评测执行官网给出的 evalscope 命令，结果见 `reports/minicpm/math_500.json` 的 score 字段。

7.3 复现注意事项

（1）防止 prefix cache 污染：benchmark 使用 seed=0，同一 serve 上连跑两次相同 input_len 的负载会产生完全相同的 prompt，第二次会大量命中 prefix cache（本队日志出现过 35.9% 命中率），使 prefill 近乎免费、总吞吐虚高 30% 以上。每个数据点都应在新启动的 serve 上获取，A/B 两侧均从冷缓存开始。

（2）C500 需要 ≥128 GB CPU 内存：在模力方舟 64 GB 内存规格实例上，纯镜像原版 `vllm serve` 也会在 inductor 编译阶段被 cgroup OOM killer 杀掉（匿名内存峰值 63.4 GB，被杀时 GPU 显存仅 858 MiB，与显存、负载、代码改动均无关）。128 GB 规格实例全程稳定。

（3）首次启动耗时：C500 需执行 GEMM autotune 预热，实测两台实例分别为 367 秒与 973 秒；第二次启动（triton cache 已存在）约 1–3 分钟。此为正常现象，不是卡死；如确需跳过可设 `VLLM_FL_MM_WARMUP=0`，但 4k 吞吐约 −6%、TTFT 变差。

（4）实例间差异：两台 C500 实例的 4k 吞吐相差 4%、TTFT 相差 66%，16k 几乎一致。报告因此以区间给数，不以单台最优值充当普遍成绩。

8 结论

本文在官方 FlagOS 栈上完成了 MiniCPM5-2B 的双平台推理优化。C500 的主要收益来自采样链融合、GEMM 路由和 prefill 同步消除；BI-V150 的主要收益来自 vendor attention 接入、发射配置调整和 paged decode attention 重写。最终，C500 的 4k 与 16k 吞吐分别达到官方基线的 2.33–2.44 倍和 1.58–1.59 倍；BI-V150 分别达到 2606.56 tok/s 和 3541.34 tok/s。MATH-500 Level 3 精度为 96.2%–98.1%，各场景 TTFT 均未回退。

测量结果也反映了实例间的差异。BI-V150 的跨时段绝对值存在漂移，因此采用同日交错 A/B 比较；两台 C500 的 4k 吞吐相差约 4%，报告保留复测区间。后续可继续研究 prefill 分块策略和编译器融合，并在更多国产芯片上检验这些实现的适用性。

参考文献

[1] 中华人民共和国国家质量监督检验检疫总局, 中国国家标准化管理委员会. GB/T 7713.3-2014 科技报告编写规则 [S]. 北京: 中国标准出版社, 2014.

[2] KWON W, LI Z, ZHUANG S, et al. Efficient memory management for large language model serving with PagedAttention[C]//Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP). 2023: 611-626.

[3] DAO T, FU D Y, ERMON S, et al. FlashAttention: fast and memory-efficient exact attention with IO-awareness[C]//Advances in Neural Information Processing Systems (NeurIPS). 2022.

[4] MADDISON C J, MNIH A, TEH Y W. The concrete distribution: a continuous relaxation of discrete random variables[C]//International Conference on Learning Representations (ICLR). 2017.

[5] FlagOS Community. vllm-plugin-FL, branch flagos-2026-s2[EB/OL]. https://github.com/flagos-ai/vllm-plugin-FL.

[6] FlagOS Community. FlagGems, tag v5.3.5[EB/OL]. https://github.com/flagos-ai/FlagGems.

[7] OpenBMB. MiniCPM5-2B[EB/OL]. https://modelscope.cn/models/OpenBMB/MiniCPM5-2B.

A 代码与实验记录索引

代码分支：插件为 `xy200303/vllm-plugin-FL` 的 `integration/vllm-plugin-fl`（C500 opt1 – opt11、天数 A1 – A7 与 A20，至 0207b7e1）；算子为 `xy200303/FlagGems` 的 `integration/flaggems`（采样融合内核、GEMM、paged_decode_attention，至 dd574fdd）。两者均为对应上游项目的正式 fork；历史开发线归档于 `archive/*`，官方基线保留于 `baseline/*`。报告与实验记录另存于 `xy200303/minicpm-flagserve` 的 `main`。

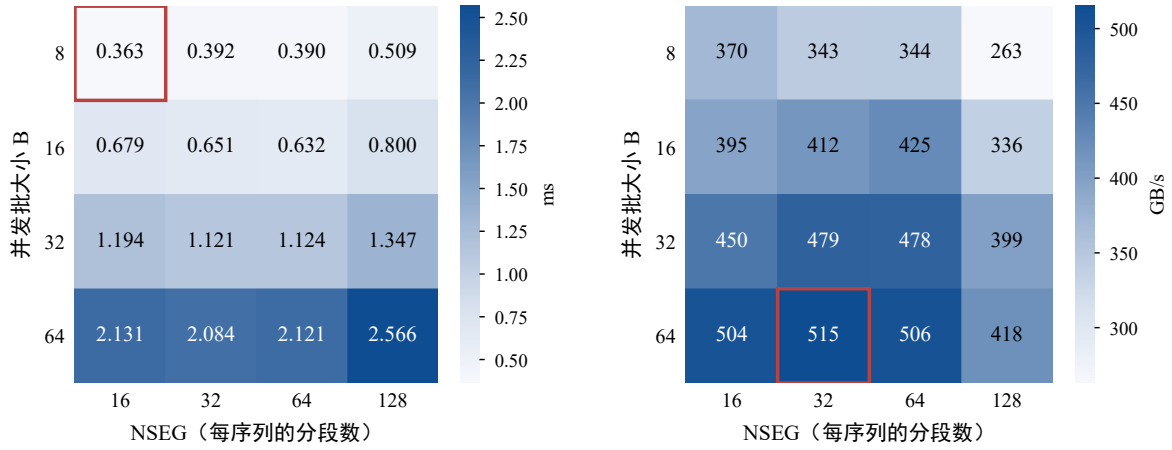
表 6 关键代码文件与报告章节的对应关系

文件（相对各自源码根目录）	作用与对应章节
`vllm_fl/dispatch/backends/vendor/metax/patches/sampler_tail_fusion.py`	C500 采样链接入，§4.2
`vllm_fl/dispatch/backends/vendor/metax/patches/linear_nn_repack.py`	nn 权重复本与形状路由，§4.3
`vllm_fl/dispatch/backends/vendor/metax/patches/mcblas_mm.py`	ctypes 直调 vendor mcBLAS，§4.3
`vllm_fl/dispatch/backends/vendor/metax/impl/attention/flash_attn.py`	prefill metadata 的当步 cumsum 缓存，§4.4
`vllm_fl/dispatch/backends/vendor/iluvatar/impl/vendor_flash_attn.py`	vendor prefill、混合步拆分、双 gather，§5.1
`vllm_fl/dispatch/backends/vendor/iluvatar/impl/decode_attn_pin.py`	3D decode 发射配置，§5.2
`vllm_fl/dispatch/backends/vendor/iluvatar/impl/decode_attn_fast.py`	A20 资格检查、slab、混合步回退，§5.3–5.4
`src/flag_gems/fused/top_p_sample.py`	直方图 top-p 与 Gumbel-Max，§4.2
`src/flag_gems/fused/paged_decode_attention.py`	A20 分段内核与 combine，§5.3
`tests/test_paged_decode_attention.py`	精度与批量不变性测试，§5.4、§6

实验记录：`docs/baselines/metax-c500/`（C500 基线与 opt1–opt11 逐项记录）、`docs/baselines/iluvatar-biv150/`（天数 A1–A9 记录）、`docs/experiments/`（后续实验、台账 `tracking.md` 与 `runs/` 原始数据）。目录中包含配置、serve 日志、benchmark CSV 和机器状态。

B 内核形状扫描与测量波动

本附录给出 paged decode attention 在 NSEG × 并发度上的实测分布，以及官方 benchmark 连续 4 次 run 的吞吐波动。数据来源分别为 `docs/experiments/a26/a26_sweep.log`（20 次重复取均值）和 `docs/baselines/metax-c500/raw_runs_baseline.csv`。



(a) 单层耗时（ms/层） (b) 有效带宽（GB/s）

图 17 NSEG × 并发度的实测分布（kv=16384）：红框为各自最优格

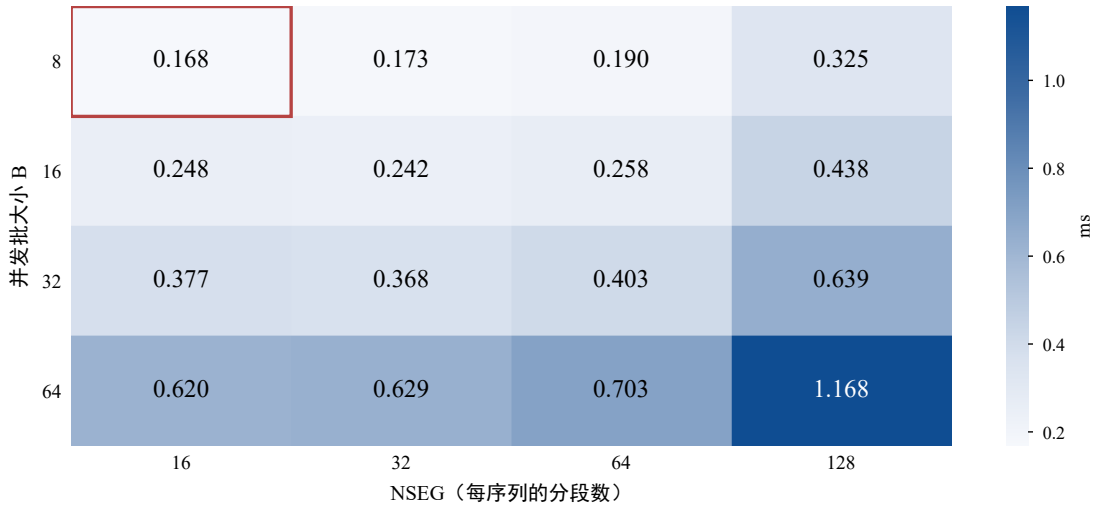


图 18 4k 场景下 NSEG × 并发度的单层耗时分布（kv=4096，ms/层）

官方 c64 形状下，NSEG=32 的有效带宽最高，为 515 GB/s，约为该卡理论带宽的 88%；继续增大 NSEG 会增加归并开销。小 batch 调整 NSEG 可获得 3%–7.4% 的收益，但官方两个

用例均为 c64，因此当前实现保留 NSEG=32。

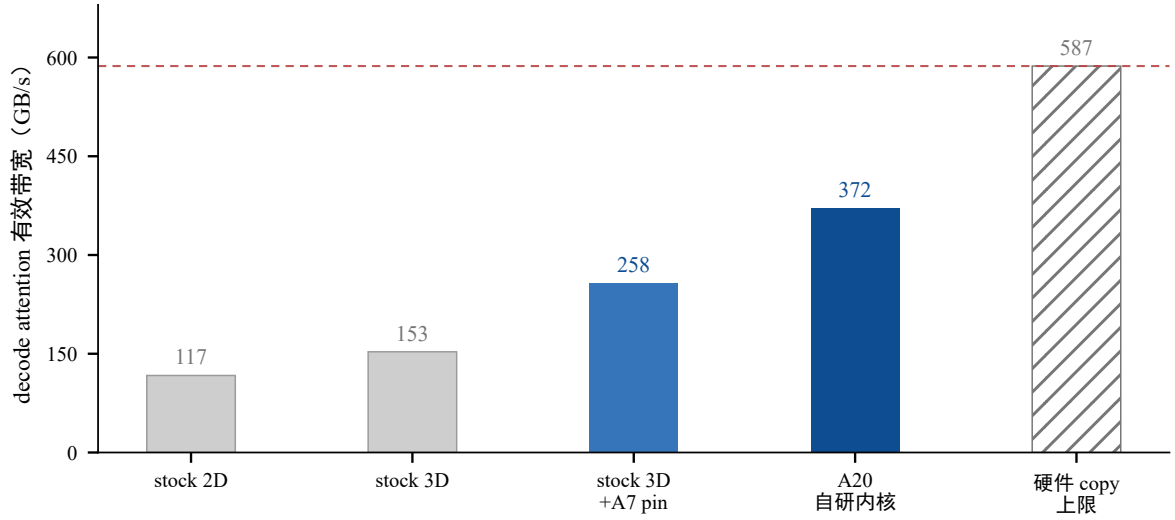


图 19 decode attention 从 stock 到自研内核的带宽演进

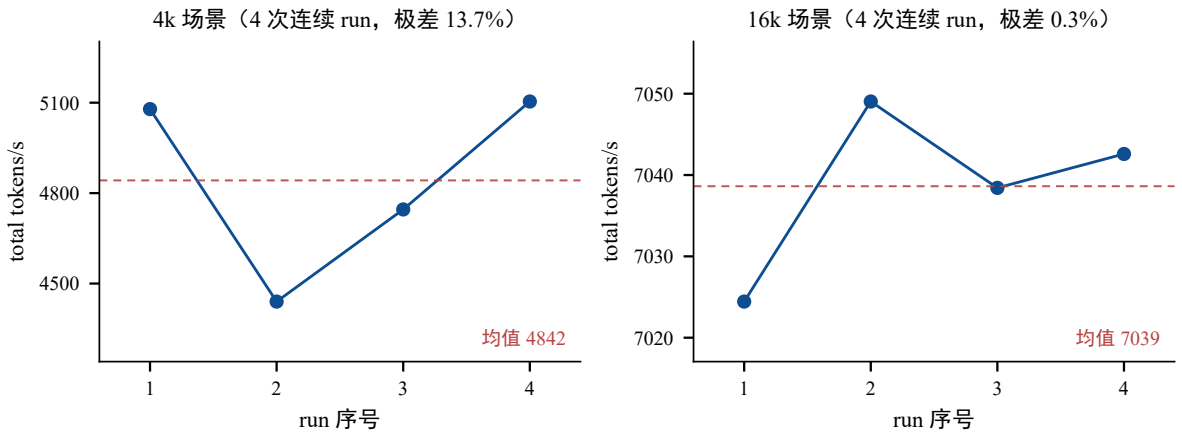


图 20 官方 benchmark 连续 4 次 run 的吞吐波动

报告中的 4k 数据取 4 次 run 的均值；16k 的连续测量极差为 0.3%，采用单次测量。优化项使用同日交错 A/B 比较，减少实例时段漂移的影响。